Les Fondations

Diagnostic du noyau

Les accès sont restaurés.

Avant de vous laisser quitter Les Fondations, Ada veut vérifier l'état du noyau central de Node Zero.

Certaines informations sont présentes, d'autres absentes, et quelques données sont corrompues.

Ada résume la situation :

> « Un bon architecte doit savoir distinguer une valeur absente, une valeur volontairement vide et une valeur invalide. »

Cette dernière mission des Fondations vous fera rencontrer :

- `undefined` ;
- `null` ;
- `NaN` ;
- `typeof` ;
- les booléens ;
- les conversions ;
- les comparaisons.

Vous réutiliserez également les notions des deux missions précédentes.

Instructions

Complétez les fonctions du fichier `diagnostic.js`.

1. Identifier le type d'une valeur

Implémentez :

```
valueType(value)
```

La fonction retourne le résultat de l'opérateur `typeof`.

```
valueType(42);
// => "number"

valueType("Nexus");
// => "string"
```

```
valueType(true);
// => "boolean"
```

2. Détecter une valeur non définie

Une variable déclarée sans recevoir de valeur contient `undefined`.

Implémentez :

```
isUndefined(value)
```

La fonction doit retourner `true` uniquement lorsque la valeur est strictement égale à `undefined`.

```
isUndefined(undefined);
// => true

isUndefined(null);
// => false
```

3. Détecter une valeur volontairement vide

`null` est utilisé lorsqu'on souhaite représenter explicitement l'absence d'une valeur.

Implémentez :

```
isNull(value)
```

```
isNull(null);
// => true

isNull(undefined);
// => false
```

4. Vérifier si une valeur est connue

Une information est considérée comme connue lorsqu'elle n'est ni `undefined`, ni `null`.

Implémentez :

```
hasKnownValue(value)
```

Exemples :

```
hasKnownValue(0);
// => true

hasKnownValue("");
// => true

hasKnownValue(null);
// => false

hasKnownValue(undefined);
// => false
```

Attention : `0`, `false` et une chaîne vide `""` sont bien des valeurs connues.

5. Vérifier si le noyau possède assez d'énergie

Implémentez :

```
hasEnoughEnergy(energy, minimumEnergy)
```

Les paramètres peuvent être des nombres ou des chaînes numériques.

Convertissez-les en nombres avant de les comparer.

```
hasEnoughEnergy("750", 500);
// => true
```

6. Vérifier l'identifiant d'une zone

Implémentez :

```
isCorrectZone(zone, expectedZone)
```

La comparaison doit être stricte.

```
isCorrectZone("FOUNDATIONS", "FOUNDATIONS");
// => true
```

Observation importante

Testez dans la console :

```
typeof null
```

Le résultat peut vous surprendre.

Vous n'avez pas besoin de corriger ce comportement : il fait partie de JavaScript.

À rechercher si nécessaire

- `JavaScript typeof`
- `JavaScript undefined`
- `JavaScript null`
- `JavaScript strict equality`
- `JavaScript Number conversion`

▶ Aides